

Object Oriented Programming



Object Oriented Programming

as its name implies it is a way of programming
oriented towards using objects

to understand what object-oriented programming
is, let's try to understand what an object is



Object

*look deep into nature, and then you
will understand everything better*

— Albert Einstein



Object

let's think about a real-life object

is an apple an object? of course, it is

what about a laptop? or a car?



Object

what if we had another laptop, same color, same brand, same specs, yet we know that this laptop is different than the first one, because each has its own **identity**

the same applies for the objects in a program, even if we have multiple **instances** of this object, it doesn't mean that all of them refer to the same object



Object

laptops can have multiple brands, come in multiple colors,
these are **attributes** that relate to this specific laptop

other laptops can have the same brand or the
same color, but still, they're different laptops



Object

a laptop can be on or off, can open a program or close it, these are **behaviors** specific to a laptop, an apple cannot behave as such, but all laptops can



Object

...all these objects are well defined ideas having their own identities separate from each other, their own attributes: information that describes their current state, and their own behaviors: things they can do

but how are these objects created in OOP?



Class

we use classes to create objects

a class describes what the object will be, but it isn't the object itself, nor a collection of objects

a class is a **blueprint** for an object, a definition



Class

if we want to build a car, before assembling the parts we draw a **schema** first, a blueprint describing everything about how that car will be built, but it isn't the car itself
we then use that blueprint to build one, or a thousand cars



Class

we define the class once and then instantiate
a thousand objects based on that class

we must write the blueprint or the class in the first place



Class

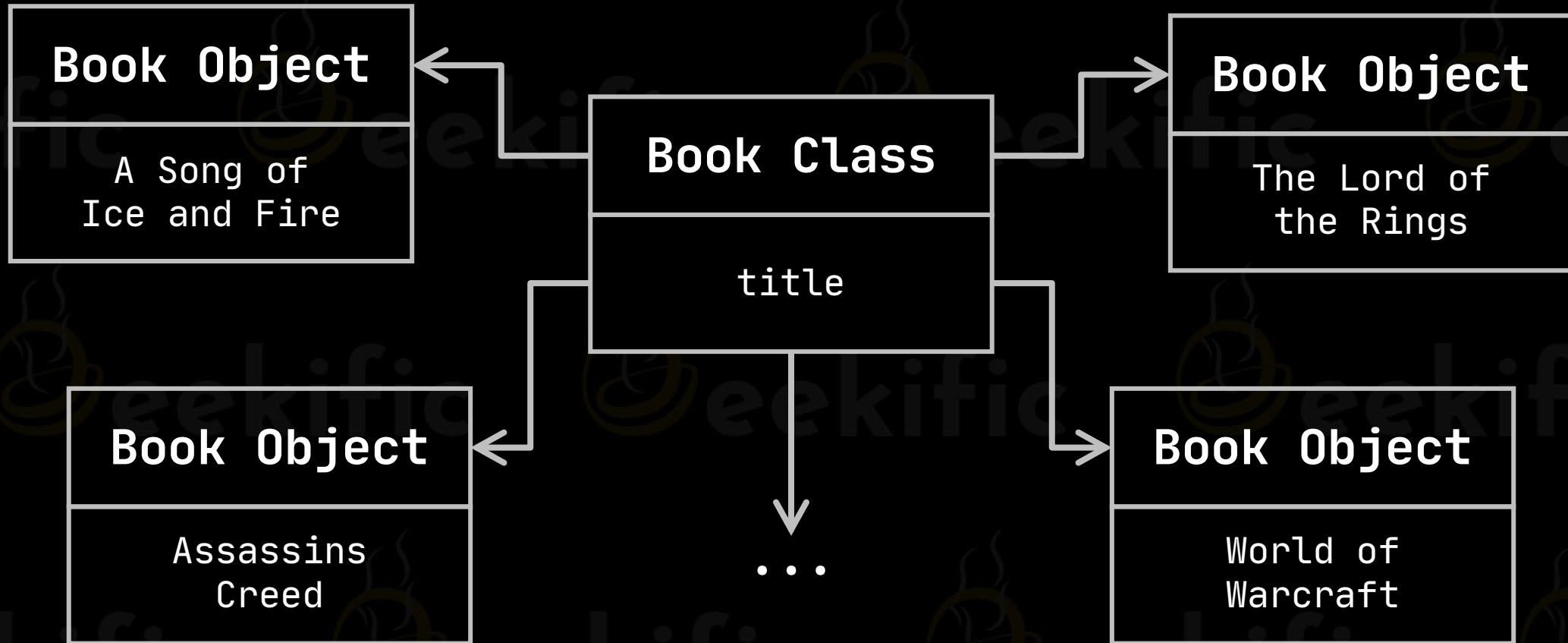
say we create a Book class with a title attribute
the class does not say what the title of the book is

it just says that books created with this class will have a title
it is just providing the definition

we can create book objects and give them the titles we want



Class



Abstraction

when we create a class, we ignore the irrelevant, the unimportant, and focus on the **essential qualities** of something rather than sticking to one specific example that illustrates our idea or a piece of information

that in OOP is referred to as: Abstraction



Abstraction



Encapsulation

while creating a class we take the information that describes our object, which are the attributes, and the things it can do, the behaviors, and **encapsulate** them, containerize them in the same unit, the same class

this in OOP is referred to as: Encapsulation.



Encapsulation

is not only about encapsulating the attributes and behaviors to keep them together, but also, to protect them and to **restrict access** to the inner workings of any object based on that class to the outside world



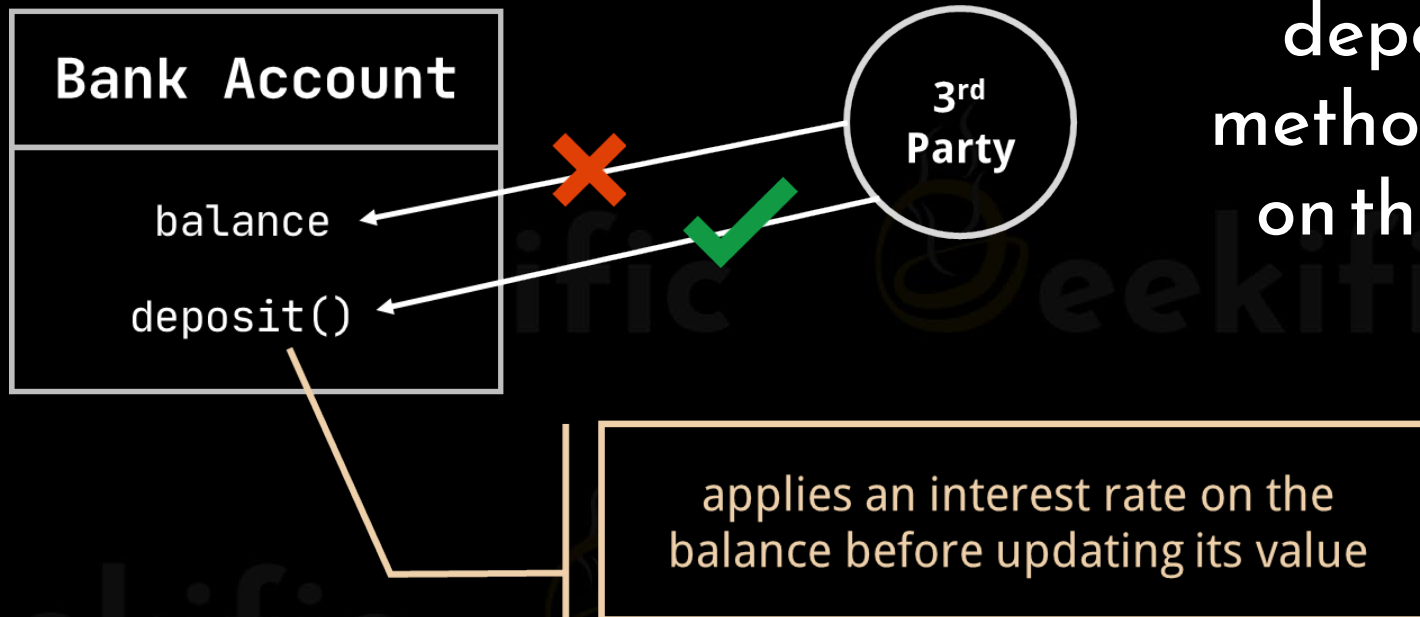
Encapsulation

the point is, an object should not reveal anything about itself, except what is **mandatory** for other parts of the application to work

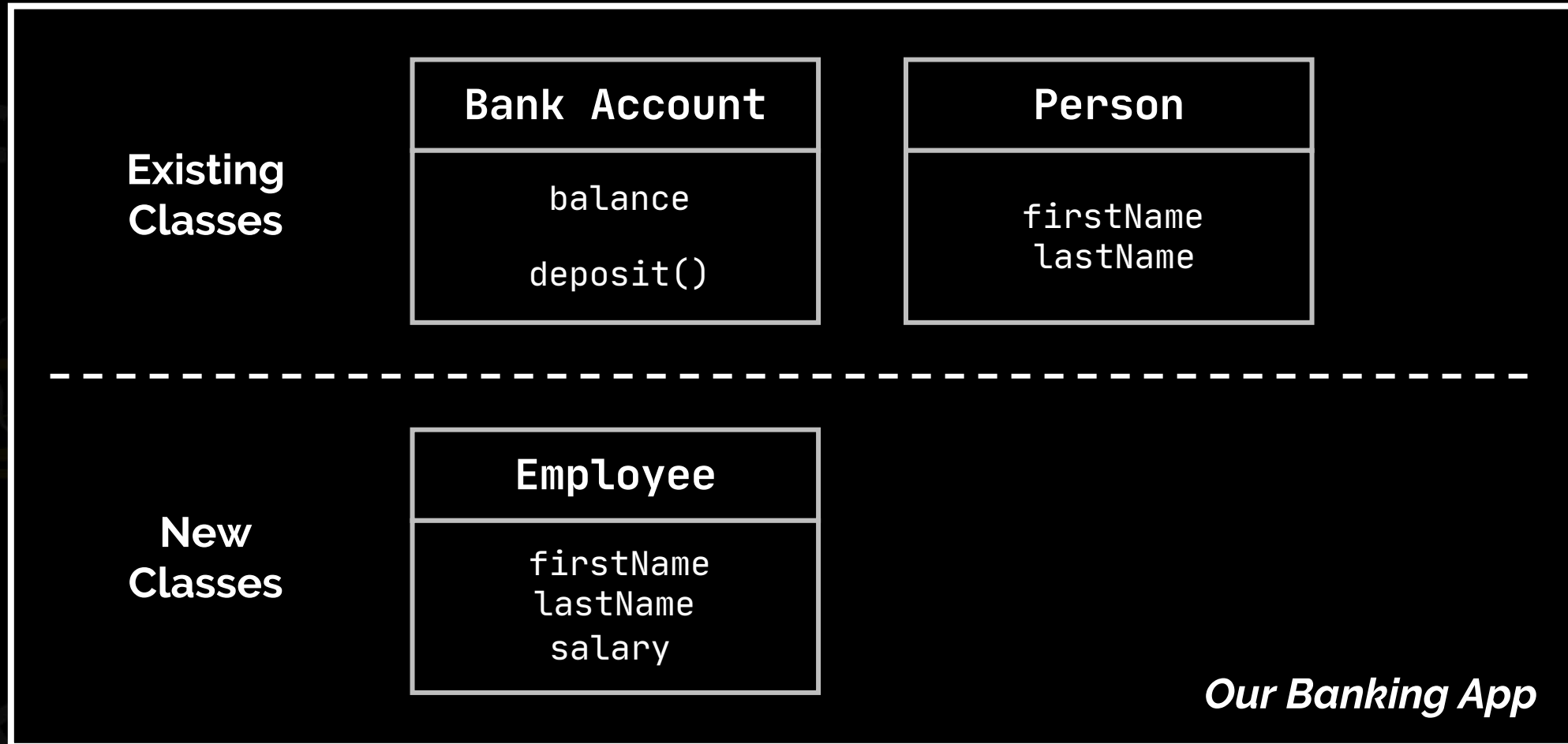


Encapsulation

we don't want anyone to access the balance without going through the deposit method, because this method is applying modifications on the value before updating it



Inheritance



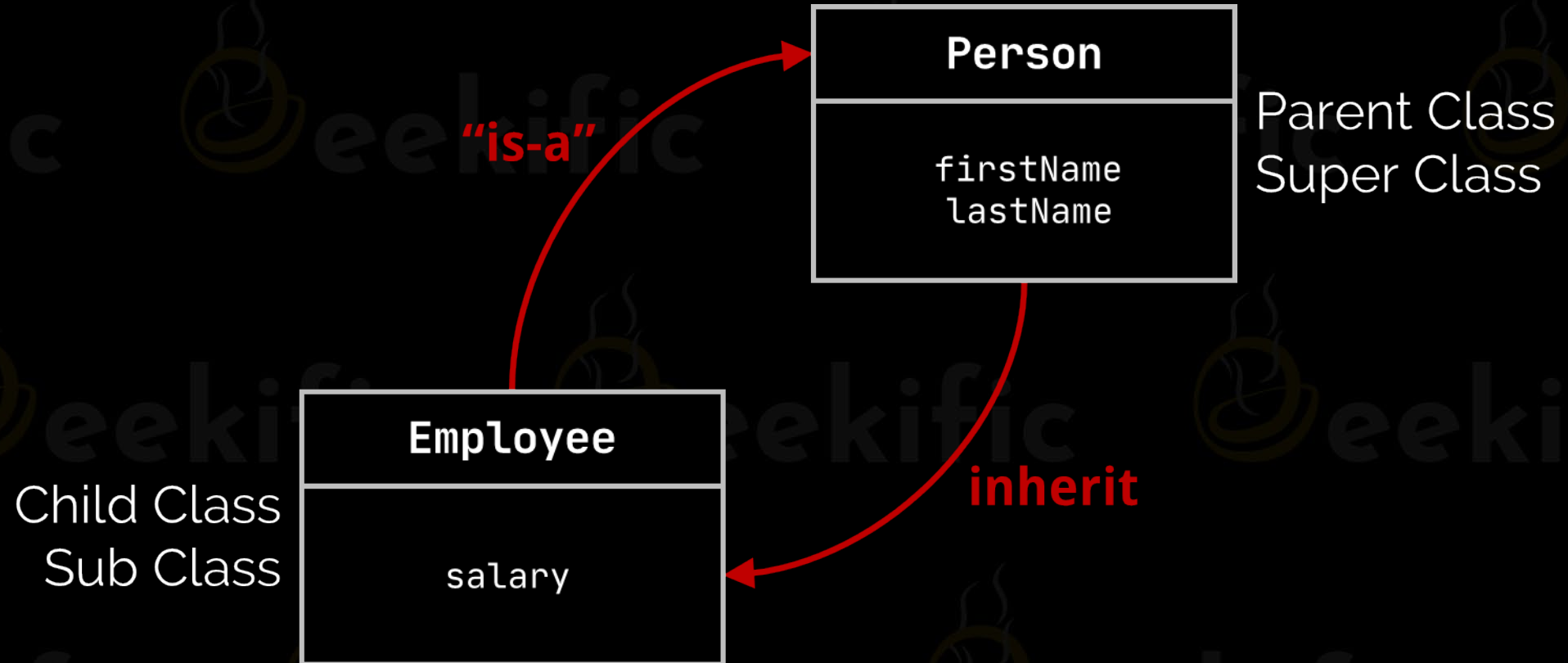
Inheritance

we would like to create an Employee class to generate objects representing each employee in the bank

while creating it we realized it contains the same attributes as the Person class, a class that already exists in our application and is being used by other parts of it



Inheritance



Inheritance

all changes made to the parent class or super class will be automatically reflected and passed down to the child class or subclass

inheritance is also referred to as **generalization** and uses an 'is-a' relationship from a specialized class to a more general class



Inheritance

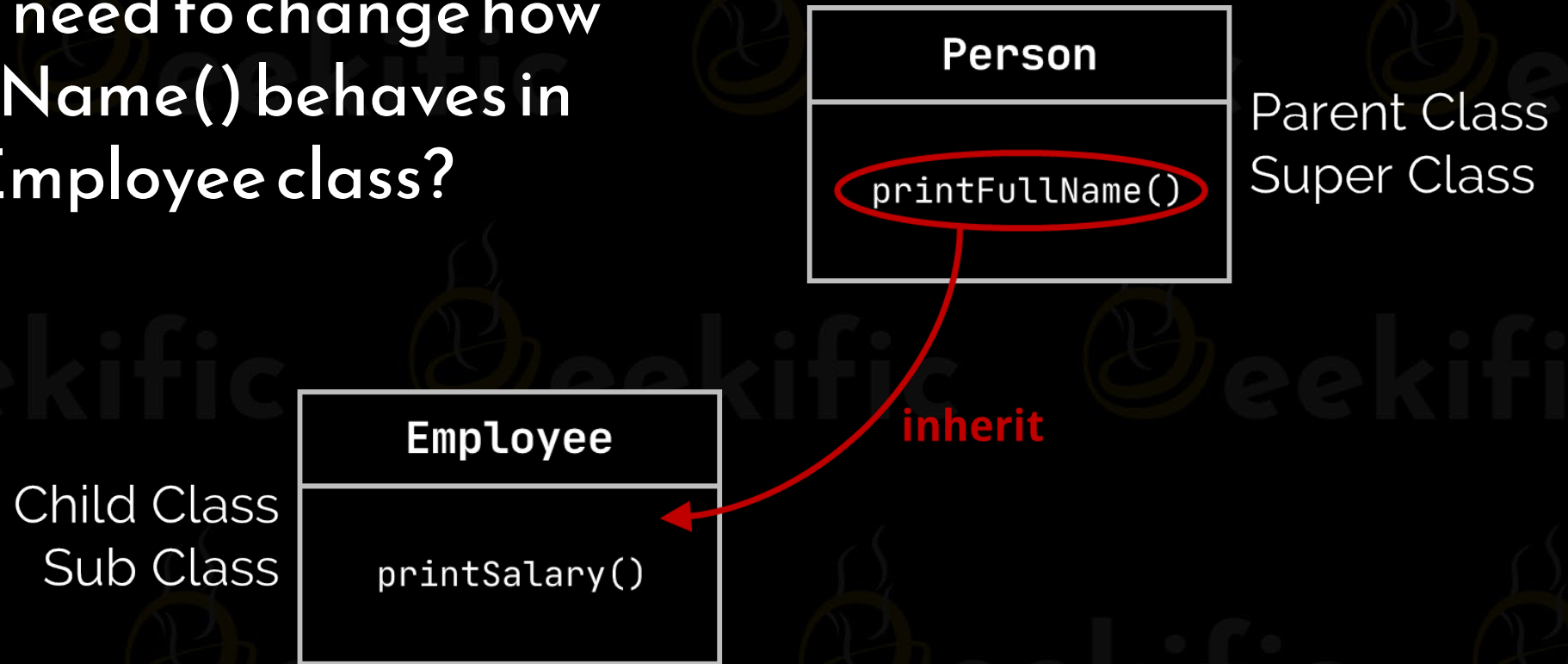
so far, inheritance was only associated with the attributes of a class, but what about its behaviors? the child class will also inherit all its parent class behaviors

if we need them as is, we are fine, and if we need to add behaviors on top of the ones present, we just add them to the child class as we did with the salary attribute



Overriding

what if we need to change how
`printFullName()` behaves in
the **Employee** class?



Overriding

what if we need to override a particular behavior with another one? OOP allows us to do so with overriding

when child classes override behaviors present in their parent classes, these behaviors are **prioritized** and executed instead of the ones present in the parent class



Polymorphism

suppose we have a list of Person objects and based on what we just learned; an Employee is also a Person

so, a list may contain Person objects, and Employee objects
if we randomly select an object from this list and invoke an overridden behavior, will it use the implementation written in the Person or Employee class?



Polymorphism

Person Object John Doe	Employee Object Emily Brown 80000	Person Object Jane Smith
Employee Object David Miller 100000	Person Object Sarah Wilson	List of Person Objects



Polymorphism

if it is a plain Person object, then it will use the implementation written in the Person class, and if it is an employee object, it will use the one written in the Employee class which is the more **specialized** behavior

this is transparent to the user, and in OOP it is referred to as: Polymorphism



Polymorphism

“many forms”

+

numbers

addition

$1 + 1 = 2$

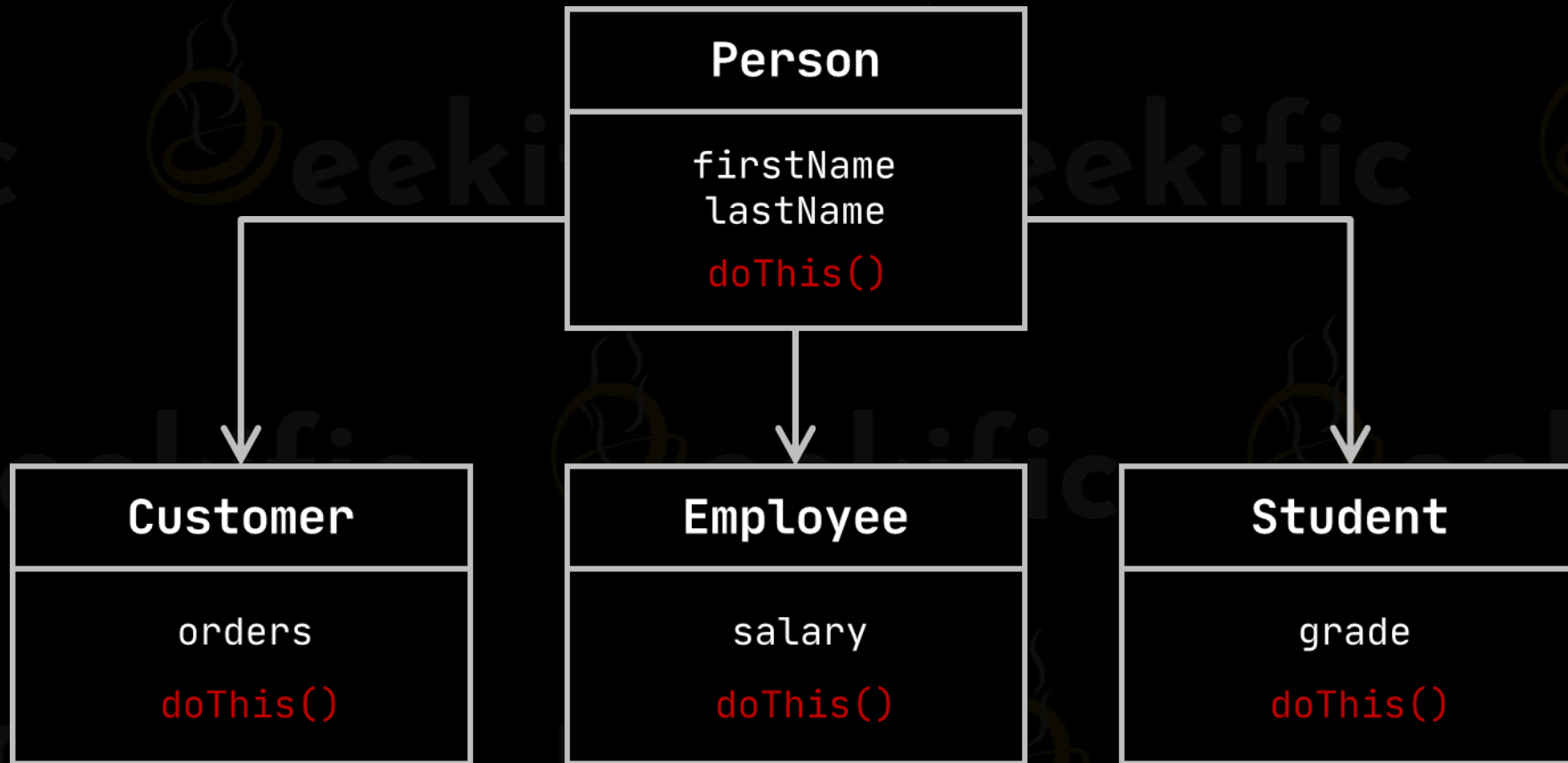
letters

concatenate

$\text{Geek} + \text{ific} = \text{Geekific}$



Polymorphism



Overloading vs Overriding

Overloading: **happens at compile-time**
method signature is different, they belong to same class

```
doSomething(String name, int score);  
doSomething(String firstName, String lastName);
```

Overriding: **happens at runtime**
method impl. is different between parent and child classes

```
parent class: doSomething(String name, int score); // diff implementation  
child class: doSomething(String name, int score); // diff implementation
```

